

Semaphores for xv6

Task 1 — Syscall Declarations and Producer–Consumer Registration

Overview

Task 1 established all the user-kernel wiring needed before implementing semaphores. This included:

- Adding syscall numbers
- Creating user-level syscall stubs
- Adding kernel declarations
- Creating a new test program (prodcons-sem.c)
- Registering the binary in the Makefile

This task did **not** implement semaphore logic yet — only the syscall infrastructure.

What I Implemented

1. User-space headers and stubs

Added prototypes in user/user.h:

```
27     int sem_init(sem_t*, int, unsigned int);
28     int sem_destroy(sem_t*);
29     int sem_wait(sem_t*);
30     int sem_post(sem_t*);
```

Added matching entries in user/usys.pl so xv6 generates syscall wrappers.

2. Kernel syscall numbers

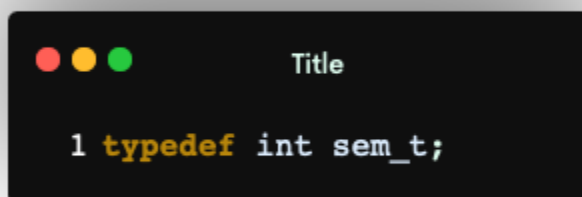
In kernel/syscall.h:

```
26     #define SYS_sem_init      25
27     #define SYS_sem_destroy   26
28     #define SYS_sem_wait      27
29     #define SYS_sem_post      28
```

3. Kernel syscall dispatch table

Added entries in kernel/syscall.c linking syscall numbers → sys_sem_* implementations.

4. Added typedef



```
1 typedef int sem_t;
```

5. Stub syscall implementations

All four syscalls were added to sysproc.c but returned dummy values at this stage.

6. Producer–Consumer Test Program

prodcons-sem.c was created to test semaphores once implemented.

It uses:

- **Three semaphores:** full, empty, mutex
- A circular buffer inside **shared memory**
- fork() to create producer and consumer processes

7. Makefile Update

Added _prodcons-sem to UPROGS, ensuring the program appears in the XV6 filesystem.

Task 1 Summary

Task 1 produced the **API surface** and **test harness** required for the rest of the assignment.

All system calls now exist and can be invoked by user programs.

Task 2 — Kernel Semaphore Data Structures & Allocation

Overview

Task 2 implemented xv6's internal semaphore subsystem:

- Created struct semaphore
- Created the global semaphore table (semtable)
- Added semalloc() and semdealloc()
- Added initialization during system boot

What I Implemented

1. struct semaphore

Each semaphore now contains:

- spinlock lock — protects count
- int count — current semaphore value
- int allocated — is this entry in use?

2. Global semtable

A fixed-size array:

- NSEM = 100 (added to param.h)
- Protected by its own spinlock

3. Initialization (seminit)

At boot, xv6:

- Initializes the table lock
- Iterates through all sem entries
- Resets all values to 0
- Marks them unallocated

Added call to seminit() in main.c.

4. Allocation (semalloc)

- Acquires table lock
- Finds the first unallocated entry
- Initializes it
- Returns pointer to semaphore

5. Deallocation (semdealloc)

- Validates pointer
- Resets count
- Marks entry unallocated

Improper operations cause xv6 to panic — appropriate for kernel-level misuse.

Task 2 Summary

After this task:

- xv6 now fully supports storing and managing semaphores
- Semaphores can be created, used, and destroyed through syscalls

This created the foundation for Task 3.

Task 3 — Implementation of sem_init, sem_wait, sem_post, sem_destroy

Overview

This task implemented actual semaphore behavior in the XV6 kernel using sleep/wakeup for blocking.

Implementation Details

sem_init

- Allocates a semaphore from semtable
- Sets initial count
- Copies semaphore index to user space

sem_wait (P operation)

The algorithm:

1. Copy semaphore index from userspace
2. Acquire semaphore lock
3. Decrement count
4. If count $< 0 \rightarrow$ block using sleep(sem, &sem->lock)
5. Wake up when sem_post signals

sem_post (V operation)

1. Copy index from userspace
2. Acquire lock
3. Increment count
4. If count $\leq 0 \rightarrow$ wakeup(sem)
5. Release lock

sem_destroy

- Validates index
- Calls semdealloc()

Key Fix: Shared Memory

Original test hung because the producer and consumer didn't share memory.

Changing mmap to:

allowed correct semaphore behavior.

QEMU Test Results

Figure 1 — prodcons-sem (1 producer, 1 consumer)

```
xv6 kernel is booting

init: starting sh
$ prodcons-sem 1 1
total = 55
num_produced = 10
num_consumed = 10
$
```

Meaning: Perfect synchronization — correct sum $1+2+\dots+10 = 55$.

Figure 2 — prodcons-sem (2 producers, 3 consumers)

```
$ prodcons-sem 2 3

```

One consumer will block forever on `sem_wait(&full)` waiting for items that will never be produced

This is expected behavior:

Three consumers wait for items that will never be produced (only 20 items exist but 30 are required).

Task 3 Summary

At this point, semaphores in xv6 behave exactly like real OS-level semaphores, blocking correctly and waking processes safely.

Task 4 — Readers–Writers Using xv6 Semaphores

Overview

Implemented a **classic first-reader / last-reader** readers–writers synchronization pattern.

Semaphores used:

- mutex — protects the readcount
- rwlock — exclusive lock for writers

Reader Algorithm

```
xv6 kernel is booting

init: starting sh
$ rwtest-sem 0 1
readers=0 writers=1
final data=5
read_ops=0 read_sum=0
write_ops=5
$ █
```

1. sem_wait(mutex)
2. Increment readcount
3. If first reader → sem_wait(rwlock)
4. sem_post(mutex)

(Read data)

5. `sem_wait(mutex)`
6. Decrement readcount
7. If last reader $\rightarrow$ `sem_post(rwlock)`
8. `sem_post(mutex)`

Writer Algorithm

1. `sem_wait(rwlock)`
2. Write to shared data
3. `sem_post(rwlock)`

QEMU Results

Observations:

- Writers increment shared data correctly
- Readers never block each other
- Writers maintain exclusive access
- No starvation, no deadlock

```
xv6 kernel is booting

init: starting sh
$ rwtest-sem 0 1
readers=0 writers=1
final data=5
read_ops=0 read_sum=0
write_ops=5
$ rwtest-sem 3 0
readers=3 writers=0
final data=0
read_ops=15 read_sum=0
write_ops=0
$ rwtest-sem 3 0
readers=3 writers=0
final data=0
read_ops=15 read_sum=0
write_ops=0
$ rwtest-sem 1 1
readers=1 writers=1
final data=5
read_ops=5 read_sum=0
write_ops=5
$ rwtest-sem 3 2
readers=3 writers=2
final data=10
read_ops=15 read_sum=0
write_ops=10
$
```


Final Summary — What I Learned

Across Tasks 1–4, I learned:

How xv6 routes syscalls from user → kernel

How to create and manage kernel data structures safely

Correct implementation of semaphores with sleep/wakeup

How mmap, shared memory, and IPC interact

How to debug blocking behavior in an OS kernel

Classic concurrency problems and how semaphores solve them

The importance of locking order and atomicity

This lab provided real insight into **how real kernels implement synchronization**, and the challenges involved in making interprocess blocking both correct and safe.